MIT SCHOOL OF COMPUTING
Rajbaug, Loni-Kalbhor, Pune

MIT-ADT UNIVERSITY
PUNE, INDIA
A Leap Towards World Class Education

Name: Advait Awase Class: SOC-08 Batch: A
Subject: PL Roll No.: 04 Exp. No.: 2
Name of the Experiment:

Performed on:
Submitted on:

Marks	Teacher's Signature with date
10	

* PL Assignment 2 *

1. List the Python sequence data types and state one key charactreristic of each with respect to mutability or ordeing.

→ Data types are :- ~~& bring~~ (str) :- Immutable and ordesed collection of chacaltevs.

- List – Mutable & ordered collection of elements
- Type – Immutable and ordeced callection of elements.

- Range :- Immutable and ordered seyuence of numbers.

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

2. Explain how indexing and slicing work in python sequence. Illustrate how they differ when applied to string & lists.

→ Indexing is used to access a single elements of a sequence using its position. positive indexing starts from -1

Slicing extracts a part of sequence using the syntax.

Difference in strings & lists:

- In strings, indexing / slicing returns characters or substrings.

- In lists, Indexing / slicing returns elements or sublists.

Example :- String :- "python" [1:4] → "yth"
List :- [1, 2, 3, 4] [1:3] → [2, 3]

3. Python program using indexing, slicing & built-in functions.

Code:-
```
text = "Engineering"
fist - char = text[0]
sub - text = text[0:5]
Length = len(text)
upper - text = text - upper text()

print(" First character", frist - (char)
print("sliced text,", sub - text)
print(" Length :", length)
print(" uppercase, ", upper text).
```

Start
↓
Input string
↓
first = text[0]
sub = text (0:5]
↓
Display auroults.
↓
stop.

4. Difference between equality (==) and identity (is).

→ • == (Equality operator) checks whether two sequences have the same values.

• is (identity operator) checks whether two variables refer to the same object in memory.

eg. a = [1, 2]
b = [1, 2] → a == b is True,
b is False. but a is b is false.

5. Evaluation of lists, sets & dictrionaries for student records.

• Lists allow duplicates and are ordered, but searching is slower.
• sets store only unique values and are fast for membership testing.

• Dictronasios store data in key-value pairs and allow fast access using keys.

Most efficent:- picronaries are best for student records because each student can be stored with a unique ID as a key, enabling fast lookup update & retrieval.

6. program to manage student data

```
students = {
    101 : "Aman",
    102 : "Riya",
    103 : "Karan",

}
print ("student with ID 101 : ", students[101]

names = list (student.values ())
print (" first two students:", names [0,2]

print ("Total students:", len (students))
```

start

↓

Create student dictionary

↓

Acess student using indexing

↓

Convert values to list and perform slicing

↓

Apply built-in function (length)

↓

Display results.

7. print first & last chasacers of a string.

→ Algorithm :- 1. start.
2. Input a string
3. print 1st chasacter using indesto
4. print last charaster using index.
5. stop.

Flowchart :-

Start
↓
Input string
↓
Get 1st & last charactor.
↓
print
↓
Stop

code :- text = input ("Enter a string:")
print ("First chacalter," text [0])
print ("last charactier;" (tex [-1])

output :- Enter a string :- sython
First charater : p
Last character :- n.

8. Creat list, hsple, set & dictionry.

→ Algorithm :-
1. start
2. Create list, huple, set, dicivision.
3. Display them.
4. stop.

Flowchart :-

start
↓
Create data
siruchiews
↓
print
↓
stop

code :-

```
lst = [1, 2, 3]
hsp = (4, 5, 6)
st = {7, 8, 9}
dct = {"a": "b": 2}.
```

Output :-

```
List : [1, 2, 3]
Tuple : (4, 5, 6)
set : {7, 8, 9}
Dicirronary : {'a': 'b' : 2}.
```

9. Program to understand indexing.

Algorithm:-
1. start
2. create a list
3. Access elements using indexing
4. print elements
5. stop.

Flowchart:-

Start
↓
Create list
↓
pecess elemenie
↓
print
↓
stop

Code: numbers = [10, 20, 30, 40,]
print ("Fist element:", numbers [0])
print ("second element", numbers [1])
print ("Last element:' numbers [-1])

Output:-
First element. 10
second element: 20
Last element: 40

10. Program to understand built-in functions.

→ Algorithm:

1. Start
2. Create a list
3. Apply built-in functions.
4. Print results.
5. Stop

Flowchart:-

Start
↓
Create list
↓
Apply functions
↓
Print
↓
End.

code:-
```
numbers = [5, 10, 15, 20]
print ("length:" len(numbers))
print ("Maximum:", max (numbers))
print ("Minimum:", min (numbers))
print ("sum:", sum (numers.))
```

output:-

length: 4
Max: 20
Min: 5.